

PROJECT: DEPLOY NIGHT

Game Night

Une vraie garde DevOps simulée, de 17h00 à 20h00.

~3 heures

Parcours obligatoire

3 étudiants

Rotation des rôles

3 + 3 incidents

3 obligatoires • 3 bonus

MISSION

A 20h00, Game Night ouvre. Le code de l'application existe déjà. Votre équipe doit construire les images, lancer la stack, diagnostiquer trois incidents réels et laisser un système compréhensible derrière elle.

STACK IMPOSEE

● Node 20 Alpine

● Nginx 1.27 Alpine

● PostgreSQL 16 Alpine

● Redis 7 Alpine

MISE EN SITUATION

17h00 - vous prenez la garde

PixelForge organise une soirée Game Night. Dans trois heures, les joueurs arrivent. La release candidate est là, mais elle n'est pas encore exploitable comme un vrai système.

17:00 Votre garde commence.

Le frontend et l'API sont codés. Vous devez les emballer proprement, comprendre la stack fournie et être capables de la remettre en ligne si quelque chose casse.

Le contexte

Pas de coach pendant le TP. Le support, les checks et votre équipe sont vos outils. Une panne n'est pas un quiz : commencez par observer.

La contrainte

Vous ne devez pas "faire marcher au hasard". Chaque correction doit partir d'un fait, d'une hypothèse puis d'un test.

OBJECTIF DE SORTIE

A 20h00 : une stack fonctionnelle, trois incidents obligatoires résolus, un snapshot DORA et un post-mortem blameless. Les bonus ne bloquent jamais la release.

ORGANISATION

Une équipe de 3, trois gardes différentes

Sur les trois incidents obligatoires, le pilote change à chaque fois. Le but est que personne ne reste spectateur et que personne ne monopolise le terminal.

Incident Commander

Décide de la prochaine action, garde la vue d'ensemble et fait formuler l'hypothèse avant toute modification.

Navigator

Lit les logs, inspecte la configuration, propose des pistes et challenge l'hypothèse.

Scribe

Note les faits, les heures, les commandes utiles et ce qui a réellement rétabli le service.

Rotation obligatoire

Incident	Commander	Navigator	Scribe
Incident 01	A	B	C
Incident 02	B	C	A
Incident 03	C	A	B

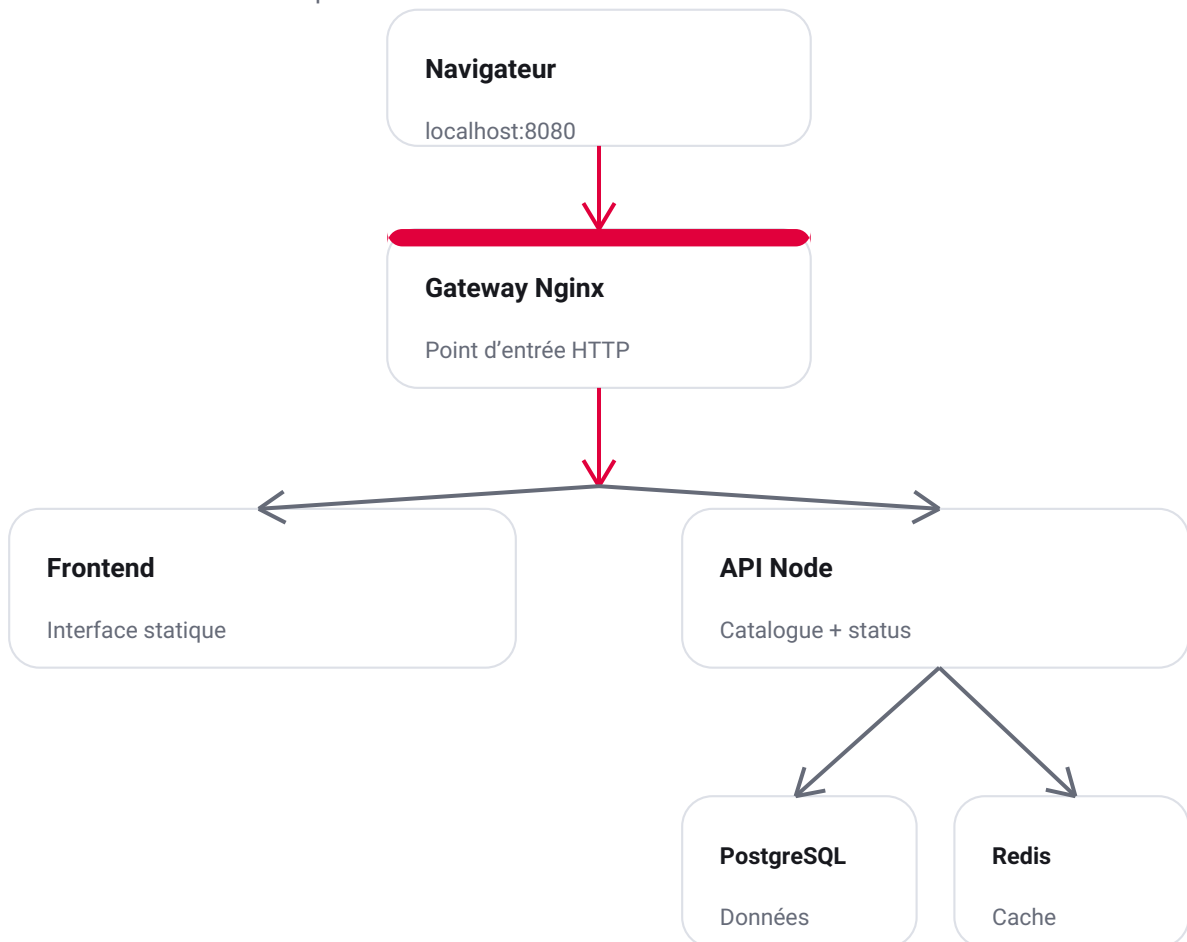
REGLE DE GARDE

Le Commander pilote, mais il ne fait pas tout. Si une personne a trouvé la solution, elle doit d'abord amener les deux autres à comprendre le signal qui l'a menée jusque-là.

ARCHITECTURE

Un seul point d'entrée, plusieurs services

Vous n'avez pas à inventer cette architecture. Elle est fournie. Votre travail est de la lire, de la lancer et de comprendre le chemin d'une requête.

**A RETENIR**

Entre containers Compose, on raisonne avec les services et leurs ports internes. Le navigateur, lui, utilise les ports publiés sur votre machine.

PAS BESOIN DE TOUT SAVOIR

Vous découvrirez certaines notions en avançant. L'important est de savoir observer la stack et expliquer le chemin de la requête.

ENVIRONNEMENT COMMUN

Les images de base sont imposées

Pas de différence de version entre les groupes. Les lignes FROM sont déjà présentes dans les Dockerfiles starter.

`node:20-alpine`**API Node**

FROM déjà fourni. A vous de construire une image qui installe les dépendances et lance server.js.

`nginx:1.27-alpine`**Frontend**

FROM déjà fourni. A vous de placer les fichiers statiques au bon endroit dans l'image Nginx.

`postgres:16-alpine`**Base de données**

Dockerfile fourni : il sert aussi à simuler un cold start pendant un incident.

`redis:7-alpine`**Cache**

Image officielle utilisée directement par Compose. Un healthcheck Redis vous sert d'exemple.

POURQUOI ?

Le TP doit tester votre raisonnement, pas votre capacité à choisir une version. Même base pour tout le monde = mêmes symptômes, mêmes checks, mêmes discussions.

PLANNING CONSEILLE

Trois heures, avec une vraie montée en pression

Le parcours obligatoire tient dans la garde. Les bonus sont là uniquement si votre équipe a de l'avance.

17:00 - 17:10

Brief + rôles + Git

17:10 - 17:35

Checkpoint 1 - image API

17:35 - 17:55

Checkpoint 2 - image frontend

17:55 - 18:20

Checkpoint 3 - stack Compose starter

18:20 - 18:40

Incident 01 - Ghost Host

18:40 - 19:05

Incident 02 - Too Fast

19:05 - 19:25

Incident 03 - Port Collision

19:25 - 19:40

Snapshot DORA

19:40 - 19:55

Post-mortem blameless

19:55 - 20:00

Checklist + go-live

BONUS

Si vous terminez tôt : Bad Gateway, Cache Phantom puis Access Denied. Ils sont volontairement un cran au-dessus et ne doivent jamais vous empêcher de terminer DORA + post-mortem.

LE STARTER

Ce que contient votre repository

Vous recevez le produit, les données, le starter Compose, les checks et les injecteurs d'incident. Vous ne partez pas d'une page blanche.

```
deploy-night-student/
├── frontend/
│   ├── Dockerfile # à compléter
│   ├── index.html
│   ├── app.js
│   └── assets/
├── api/
│   ├── Dockerfile # à compléter
│   ├── package.json
│   └── server.js
├── db/ # fourni
├── gateway/ # fourni
├── compose.yaml # starter
├── checks/
├── ops/
├── tools/
└── incident
```

Ce que vous construisez

Deux images applicatives propres, puis une stack que vous savez expliquer et réparer. Les services techniques complexes sont déjà câblés pour ne pas transformer le PLD en tuto Compose.

Ce que vous ne faites pas

Pas de développement du produit. Pas de redesign du frontend. Pas de chasse aux scripts incidents. Votre énergie va dans Docker, Compose, Git et le diagnostic.

AVANT LE PREMIER COMPOSE UP

Lancez `./tools/prepare-runtime.sh`. Il prépare la configuration runtime du gateway et crée le fichier `.env` s'il manque. Le launcher `./incident` choisira ensuite automatiquement le bon binaire Linux/macOS et gère la quarantaine macOS si nécessaire.

GARDE-FOU GIT

Git : votre première conversation propre

Le TP ne refait pas l'exercice Pull Request complet. En revanche, votre historique doit raconter ce que votre équipe a réellement fait.

```
git init
git add .
git commit -m "chore: initialize deploy night starter"
```

- Faites de petits commits après un checkpoint ou une correction comprise.
- Utilisez au moins trois types de Conventional Commits sur le TP : par exemple chore, fix, docs.
- Ne committez jamais .env, les états d'incident ou des secrets.
- Si vous utilisez des branches, gardez-les courtes et mergez uniquement une correction testée.

QUESTION DE GROUPE

Si un incident vous oblige à faire trois essais, devez-vous commiter les trois essais ? Discutez de ce qu'un historique Git utile devrait garder - et de ce qu'il peut laisser disparaître.

CHECKPOINT 1 • 25 MIN

Containeriser l'API

Le code Node est prêt. Votre mission est uniquement de fabriquer une image reproductible qui sait lancer cette API.

```
FROM node:20-alpine

# La suite est à vous.
```

Avant de coder, regardez deux fichiers

- package.json : quelles dépendances ? quelle commande de démarrage ?
- server.js : quel port écoute réellement l'application ? quelles variables d'environnement lit-elle ?

Votre Dockerfile doit atteindre ces résultats

- un dossier de travail clair dans le container ;
- les dépendances installées dans l'image ;
- le code présent au runtime ;
- le port applicatif documenté ;
- une commande de démarrage qui garde le process principal au premier plan.

CHECK

Quand vous pensez avoir terminé : `./checks/check-01-api.sh`. Le check valide votre recette d'image ; la connexion PostgreSQL viendra avec la stack.

CHECKPOINT VISUEL

Une image, pas encore un système

A ce stade, ne cherchez pas à faire fonctionner toute l'application. Vous devez simplement pouvoir construire l'image API de façon fiable.

```
$ ./checks/check-01-api.sh
```

```
[OK] image API construite
```

```
[OK] Dockerfile interprété et image exécutable
```

```
Checkpoint validé.
```

A EXPLIQUER

Quelle différence faites-vous entre une image et un container ? Pourquoi l'installation des dépendances doit-elle être décrite dans le Dockerfile ?

NE VOUS PRECIPITEZ PAS

Si l'API seule quitte parce qu'aucune base n'est présente, ce n'est pas encore le problème du checkpoint. Le système complet arrive ensuite.

CHECKPOINT 2 • 20 MIN

Containeriser le frontend

Le frontend est statique. Pas de build JavaScript : il faut simplement le servir proprement avec l'image Nginx imposée.

```
FROM nginx:1.27-alpine

# La suite est à vous.
```

- Repérez les fichiers réellement nécessaires au navigateur : HTML, CSS, JavaScript et images.
- Trouvez le répertoire servi par défaut par l'image Nginx officielle au lieu de le deviner.
- Copiez les fichiers à cet endroit et documentez le port interne réellement utilisé par Nginx.

QUESTION

Pourquoi le frontend n'a-t-il pas besoin de Node au runtime ici ? Et qu'est-ce que cela change dans la taille ou la responsabilité de l'image ?

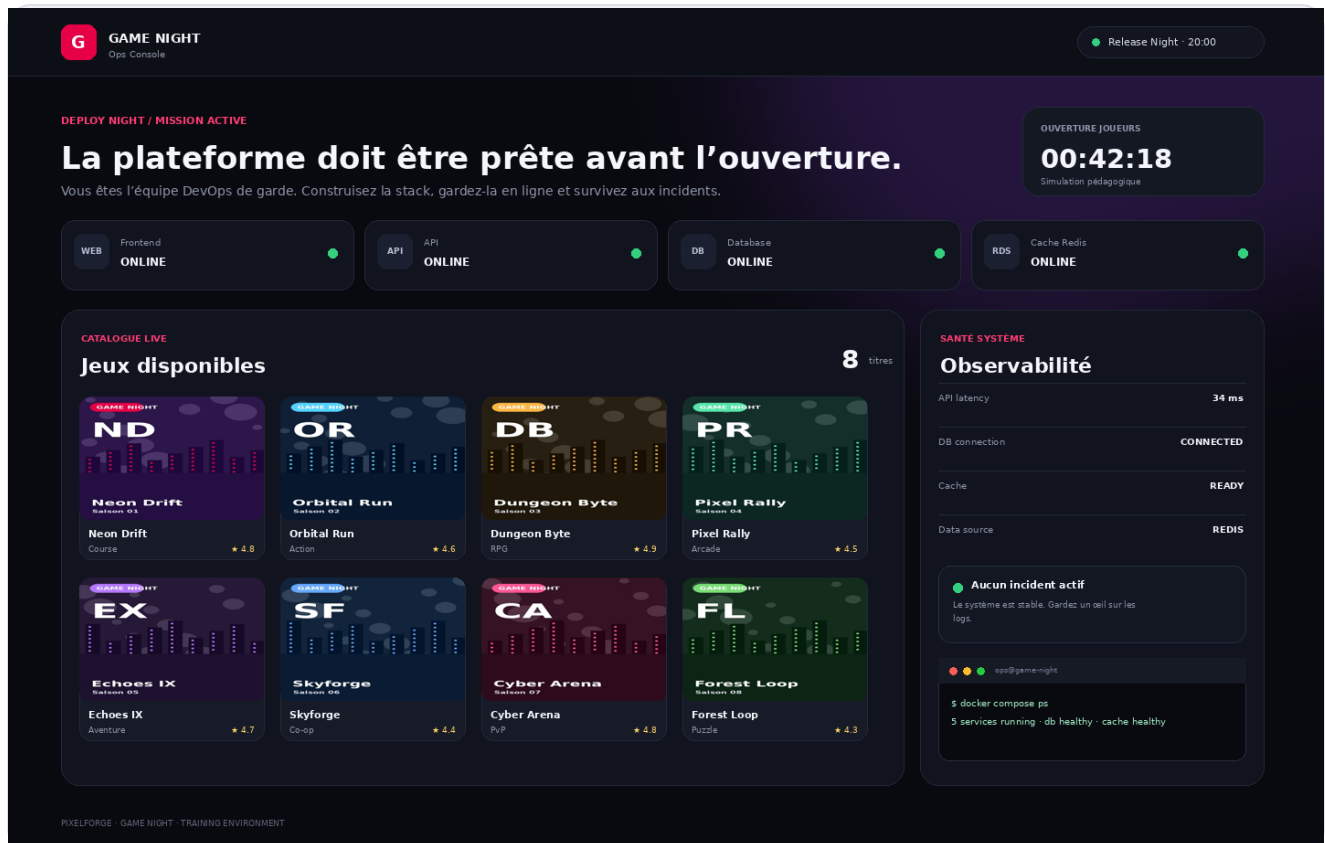
CHECK

Validez avec `./checks/check-02-frontend.sh`. Le script lance votre image sur un port temporaire et vérifie que Game Night, son CSS, son JavaScript et un visuel sont réellement servis - pas la page Nginx par défaut.

CHECKPOINT VISUEL

Le produit existe déjà - vous l'emballez

Le design n'est pas le sujet du TP. Cette interface est fournie pour que chaque action Docker ou Compose ait un effet visible et compréhensible.



Référence Game Night - l'interface complète, affichée sans recadrage.

A CE STADE

Votre frontend seul peut s'afficher, mais les données live viennent de l'API. La prochaine étape consiste à faire vivre tous les services ensemble.

COMPOSE • BASES

Compose : vous ne partez pas d'une page blanche

Le starter contient déjà les cinq services et les variables de l'application. Vous devez comprendre les blocs, pas recopier un gros YAML depuis le support.

service

une responsabilité

build / image

comment obtenir l'image

ports

hôte : container

environment

configuration runtime

depends_on

ordre de démarrage

volumes

données persistantes

UN EXEMPLE EST DEJA DANS LE STARTER

Redis possède déjà un healthcheck et l'API attend son état healthy. Ne copiez rien maintenant : observez simplement ce mécanisme. Il pourrait vous servir plus tard.

CHECKPOINT 3 • 25 MIN

Assembler la stack de base

Vos deux images sont prêtes. Utilisez maintenant le `compose.yaml` starter pour lancer le système complet.

- Lisez chaque service et dites à voix haute son rôle.
- Repérez les deux services construits depuis vos Dockerfiles et les services basés directement sur une image officielle.
- Repérez les ports publiés vers votre machine. Les autres communications restent internes à la stack.
- Repérez les variables `DB_HOST`, `REDIS_HOST` et les noms de services correspondants.
- Lancez `./tools/prepare-runtime.sh`, puis construisez et démarrez la stack.

```
docker compose config
docker compose up -d --build
docker compose ps
```

CHECK

Quand Game Night répond via le gateway : `./checks/check-03-stack.sh`. Ne rajoutez pas de complexité “pour faire pro” : vous aurez bientôt une raison concrète de faire évoluer Compose.

CHECKPOINT VISUEL

18h20 - Game Night est en ligne

Avant le premier incident, prenez trente secondes pour observer l'état sain. Vous aurez besoin de savoir à quoi ressemble le système quand tout va bien.

The screenshot shows the 'GAME NIGHT Ops Console' dashboard. At the top, it says 'DEPLOY NIGHT / MISSION ACTIVE' and 'Release Night - 20:00'. A large banner reads 'La plateforme doit être prête avant l'ouverture.' with a subtext: 'Vous êtes l'équipe DevOps de garde. Construisez la stack, gardez-la en ligne et survivez aux incidents.' To the right, a timer shows 'OUVERTURE JOUEURS 00:42:18' for a 'Simulation pédagogique'. Below the banner, four status cards are shown: 'WEB Frontend ONLINE', 'API API ONLINE', 'DB Database ONLINE', and 'Cache Redis ONLINE', all with green status indicators. The 'CATALOGUE LVE' section displays 'Jeux disponibles' (8 titres) in a grid of 8 game cards: Neon Drift (Course, 4.8), Orbital Run (Action, 4.6), Dungeon Byte (RPG, 4.9), Pixel Rally (Arcade, 4.5), Echoes IX (Aventure, 4.7), Skyforge (Co-op, 4.4), Cyber Arena (PvP, 4.8), and Forest Loop (Puzzle, 4.3). On the right, the 'SANTÉ SYSTÈME' section shows 'Observabilité' metrics: API latency (34 ms), DB connection (CONNECTED), Cache (READY), and Data source (REDIS). A green dot indicates 'Aucun incident actif' with the note 'Le système est stable. Gardez un œil sur les logs.' Below this, a terminal snippet shows '\$ docker compose ps' and '\$ services running · db healthy · cache healthy'.

Etat sain attendu : frontend, API, PostgreSQL et cache joignables.

BASELINE

Notez votre URL, la sortie de `docker compose ps` et un `curl /api/status`. Une bonne prise d'incident commence par savoir comparer avec l'état normal.

INCIDENT MODE

Faits -> hypothèse -> test -> correction

Le script injecte la panne, mais il ne donne pas la réponse. Votre méthode doit rester la même sur les trois incidents.

1

Faits

Qu'est-ce qui est cassé exactement ? Quels services sont up, exited, healthy ?

2

Observer

Logs, ps, curl, configuration. Cherchez un signal précis, pas une intuition.

3

Hypothèse

Formulez une cause en une phrase avant de modifier le système.

4

Test

Faites le plus petit test qui peut confirmer ou invalider l'hypothèse.

5

Corriger

Changez une chose, relancez ce qui est nécessaire et utilisez le check.

COMMANDES AUTORISEES

Tout ce que vous savez déjà : docker compose ps, logs, exec, config, curl, votre éditeur, Git. Le bon outil est celui qui répond à votre question du moment.

INCIDENT 01 • OBLIGATOIRE

Ghost Host - prise d'astreinte

Votre premier Incident Commander prend la main. Le système fonctionnait il y a une minute. Une configuration vient d'être déployée.

```
./incident 1  
  
# Laissez le script terminer. Puis observez.
```

IMPACT UTILISATEUR

Le catalogue ne charge plus. L'API n'arrive plus à établir sa connexion PostgreSQL et quitte au démarrage.

Votre mission

- identifier le premier signal utile ;
- localiser la frontière de panne ;
- formuler la cause avant de corriger ;
- rétablir le service sans modifier le code applicatif ;
- valider avec `./checks/check-incident-01.sh`.

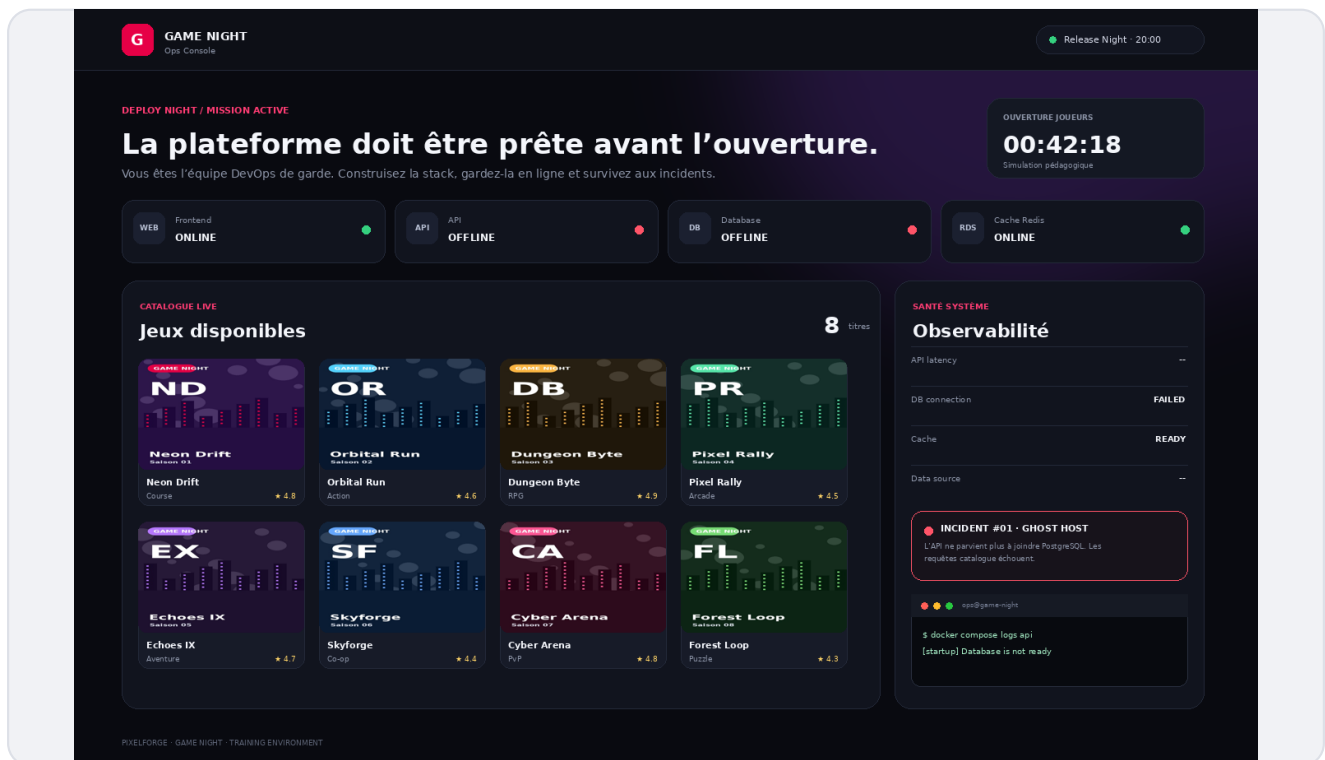
A NOTER POUR LE POST-MORTEM

Heure de début, symptôme utilisateur, premier log utile, hypothèse retenue et heure de rétablissement. Pas de nom de coupable.

INCIDENT 01 • INDICES

Ghost Host - indices progressifs

N'ouvrez un indice que si le groupe tourne réellement en rond. Un indice doit relancer votre raisonnement, pas remplacer le diagnostic.



Référence visuelle de la plateforme pendant l'incident 01.

Indice 1

Quel service quitte ?
Commencez par ses logs.

Indice 2

Repérez précisément le host et
le port vers lesquels l'API essaie
de se connecter.

Indice 3

Dans un réseau Compose,
comment un service trouve-t-il
un autre service ?

INCIDENT 02 • OBLIGATOIRE

Too Fast - démarré ne veut pas dire prêt

Rotation des rôles. Un cold start est déclenché. La base met volontairement du temps avant d'accepter les connexions.

```
./incident 2  
  
# Le script effectue un redémarrage à froid.
```

IMPACT UTILISATEUR

La stack "démarré", mais l'API part trop tôt, échoue sur PostgreSQL puis quitte. Le problème apparaît surtout lors d'un redémarrage complet.

Le défi

- prouver que le problème vient du timing, pas d'un mauvais mot de passe ou d'un mauvais host ;
- faire distinguer à Compose service démarré et service prêt ;
- rendre les prochains cold starts fiables ;
- valider avec `./checks/check-incident-02.sh`.

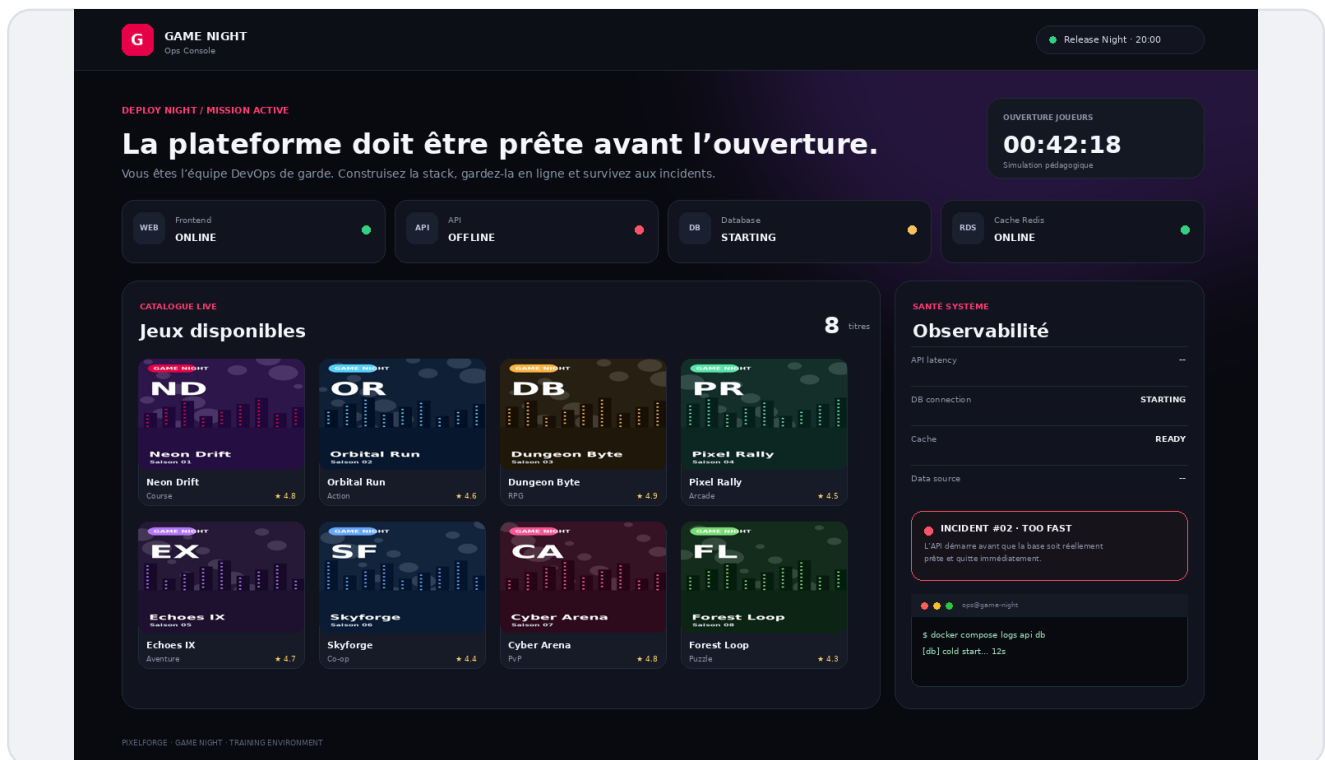
PISTE PEDAGOGIQUE

Votre starter contient déjà un service qui sait annoncer son état de santé. Observez comment il le fait avant d'inventer autre chose.

INCIDENT 02 • INDICES

Too Fast - indices et validation

Le but n'est pas d'apprendre une syntaxe par cœur. Il faut comprendre quelle information manque à l'orchestrateur.



Référence visuelle de la plateforme pendant l'incident 02.

Indice 1

Comparez le moment où le container DB apparaît et le moment où PostgreSQL accepte réellement une requête.

Indice 2

Le service Redis du starter possède déjà un mécanisme de santé. Lisez-le.

Indice 3

Comment demander à l'API d'attendre un état "healthy" plutôt qu'un simple "started" ?

INCIDENT 03 • OBLIGATOIRE

Port Collision - la release ne démarre plus

Dernière rotation obligatoire : le troisième membre devient Incident Commander. Cette fois, la panne se produit avant même que l'application puisse répondre.

```
./incident 3  
  
# Lisez l'erreur de déploiement jusqu'au bout.
```

IMPACT UTILISATEUR

La nouvelle release ne peut pas monter correctement : une ressource de votre machine hôte est déjà occupée.

- identifier quelle ressource entre en collision ;
- distinguer le port interne du container et le port publié sur votre machine ;
- expliquer pourquoi EXPOSE n'est pas l'équivalent d'un mapping ports ;
- rétablir la release puis lancer `./checks/check-incident-03.sh`.

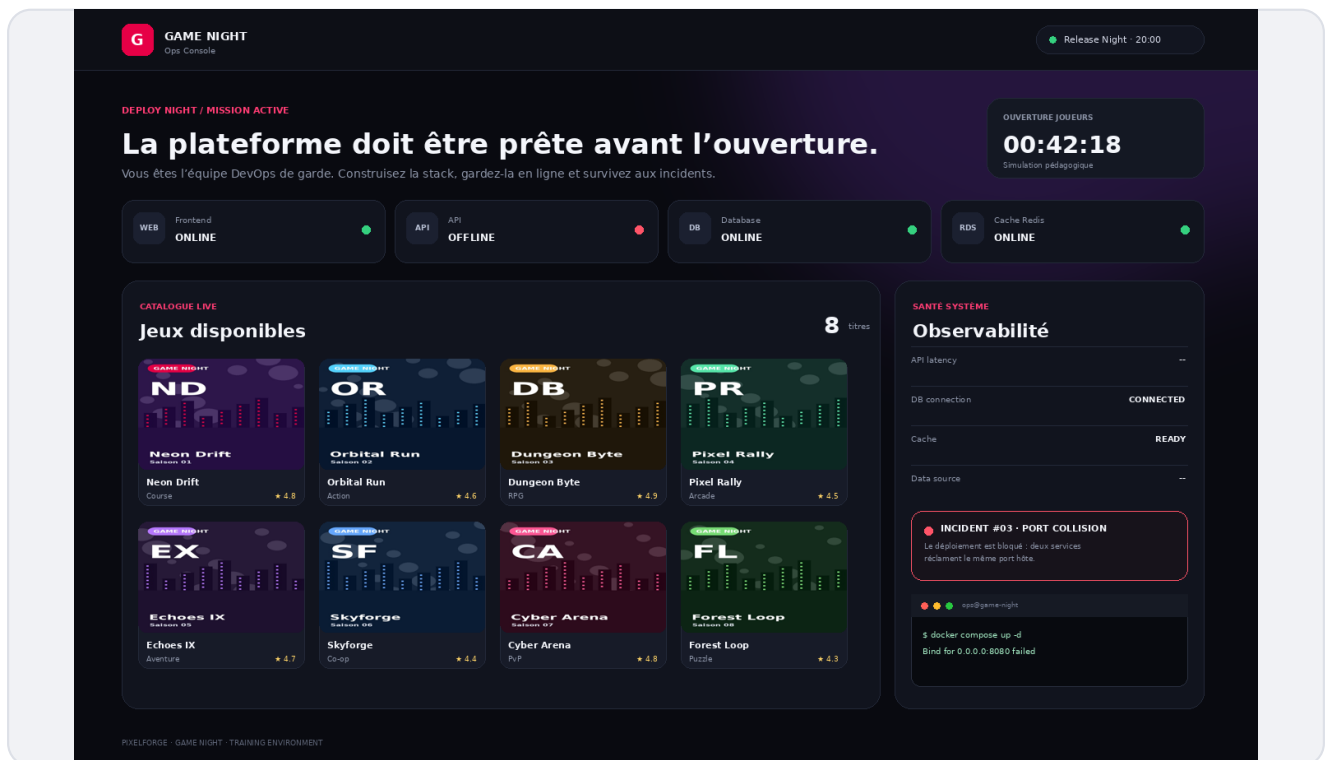
FIN DU PARCOURS INCIDENT OBLIGATOIRE

Quand le check passe, arrêtez-vous. Avant les bonus, votre équipe doit faire DORA puis écrire un post-mortem sur l'un des trois incidents.

INCIDENT 03 • INDICES

Port Collision - indices et validation

Ici, le message d'erreur est votre meilleur outil. Ne commencez pas par éditer Compose : commencez par lire.



Référence visuelle de la plateforme pendant l'incident 03.

Indice 1

Cherchez "allocated", "bind" ou "port" dans la sortie Docker.

Indice 2

Listez les ports publiés sur l'hôte par vos services.

Indice 3

Deux containers peuvent écouter le même port interne. Le conflit apparaît où ?

DORA • 15 MIN

Mesurer la garde, pas juste “ça marche”

Après les trois incidents obligatoires, utilisez les données de release fournies et vos propres temps de rétablissement.

```
python3 tools/dora-report.py
```

```
# Le script affiche les données brutes. A vous de nommer, calculer et classer.
```

Deployment Frequency

Combien de déploiements sur la période ?

Lead Time for Changes

Combien de temps entre merge et production ?

Change Failure Rate

Quelle part des déploiements provoque un incident ?

Time to Restore

Combien de temps pour rétablir après incident ?

A RENDRE DANS ops/DORA.md

Pour chaque métrique : votre valeur ou estimation, votre calcul en une phrase, puis son classement throughput ou stability. Ajoutez enfin la métrique la plus directement touchée par chacun de vos trois incidents.

POST-MORTEM • 15 MIN

Chercher la cause, pas le coupable

Choisissez l'un des trois incidents obligatoires. Le fichier ops/POSTMORTEM.md est déjà prêt à compléter.

Timeline

Début, détection, décision, rétablissement.
Uniquement des faits observables.

Impact

Ce que l'utilisateur ne pouvait plus faire - ou ce qui était dégradé.

Cause système

La configuration ou le mécanisme qui a permis l'incident. Aucun prénom.

3 actions

Des actions qui réduisent la probabilité, l'impact ou le temps de rétablissement.

DORA

Quelle métrique a été dégradée et pourquoi ?

BLAMELESS

"X a mis le mauvais host" n'est pas une cause système. "La configuration pouvait être déployée sans validation de connectivité" ouvre déjà une vraie piste d'amélioration.

BONUS 04 • OPTIONNEL

Bad Gateway - tout est up, mais rien ne passe

Vous avez terminé DORA et le post-mortem ? Vous pouvez reprendre la garde. Le premier bonus vous oblige à suivre le chemin HTTP de bout en bout.

```
./incident 4

# Cette panne touche le gateway runtime.
```

IMPACT

Le frontend peut charger, les containers peuvent sembler démarrés, mais les appels vers /api répondent 502 Bad Gateway.

- prouver si l'API est réellement down ou seulement inaccessible via le gateway ;
- suivre Browser -> Gateway -> API ;
- inspecter la configuration Nginx runtime ;
- valider avec ./checks/check-incident-04-bonus.sh.

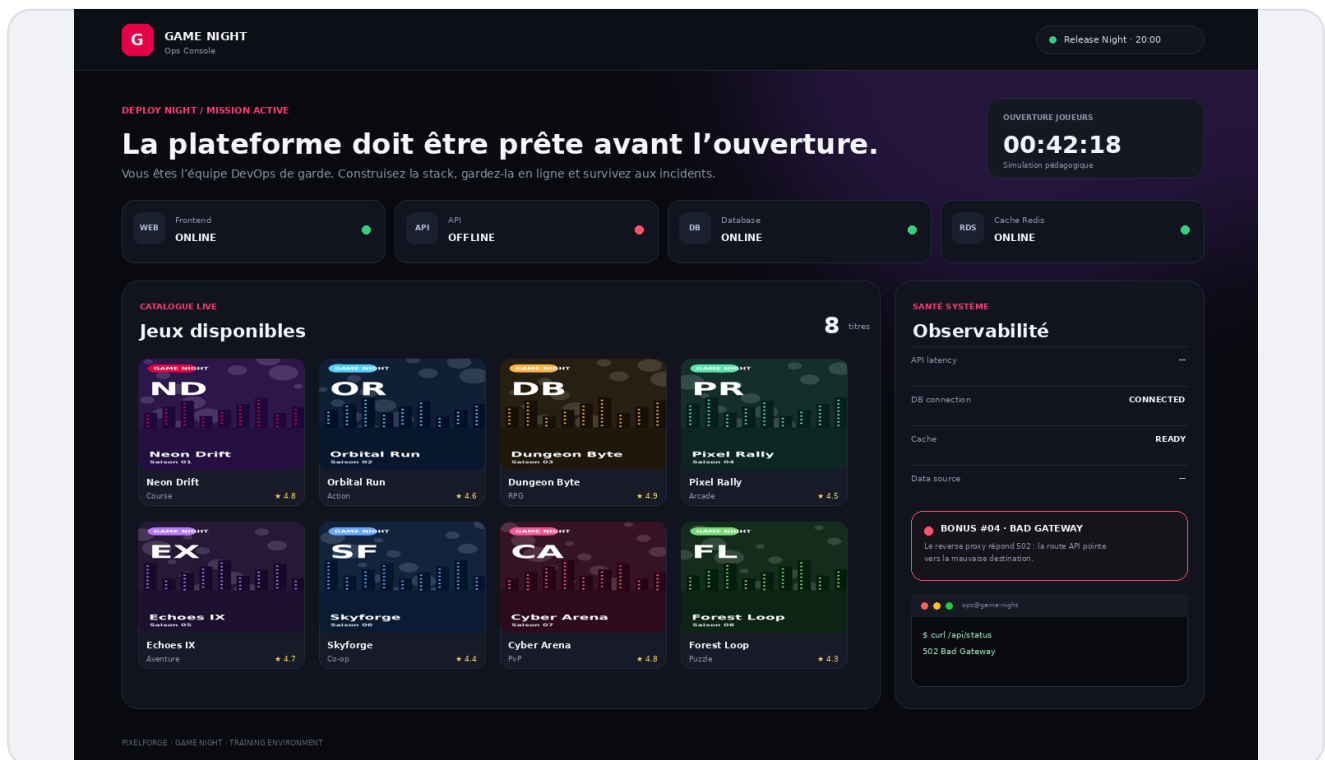
QUESTION BONUS

Pourquoi docker compose ps peut-il être entièrement rassurant alors que l'utilisateur reçoit quand même une erreur 502 ?

BONUS 04 • INDICES

Bad Gateway - indices

Le piège est de confondre “process en cours” et “chemin de requête correct”.



Référence visuelle de la plateforme pendant Bad Gateway.

Indice 1

Comparez un appel direct à l'API et le même appel via le gateway.

Indice 2

Regardez le fichier réellement monté dans Nginx, pas seulement le fichier source.

Indice 3

Le port cible de proxy_pass doit correspondre au port interne écouté par l'API.

BONUS 05 • OPTIONNEL

Cache Phantom - le système marche... moins bien

Cette fois, il n'y a pas de panne totale. Le challenge est justement de détecter une dégradation : l'API doit rester disponible grâce au fallback PostgreSQL pendant que Redis est hors circuit.

```
./incident 5

# La plateforme peut continuer à répondre.
```

IMPACT

Redis disparaît du chemin. L'API retombe sur PostgreSQL et le catalogue reste disponible, mais l'architecture n'est plus dans son état nominal.

- vérifier d'abord que l'API reste disponible, puis observer le statut du cache et la source des données ;
- vérifier les logs de connexion Redis ;
- retrouver comment l'API localise le service de cache ;
- rétablir Redis puis valider avec `./checks/check-incident-05-bonus.sh`.

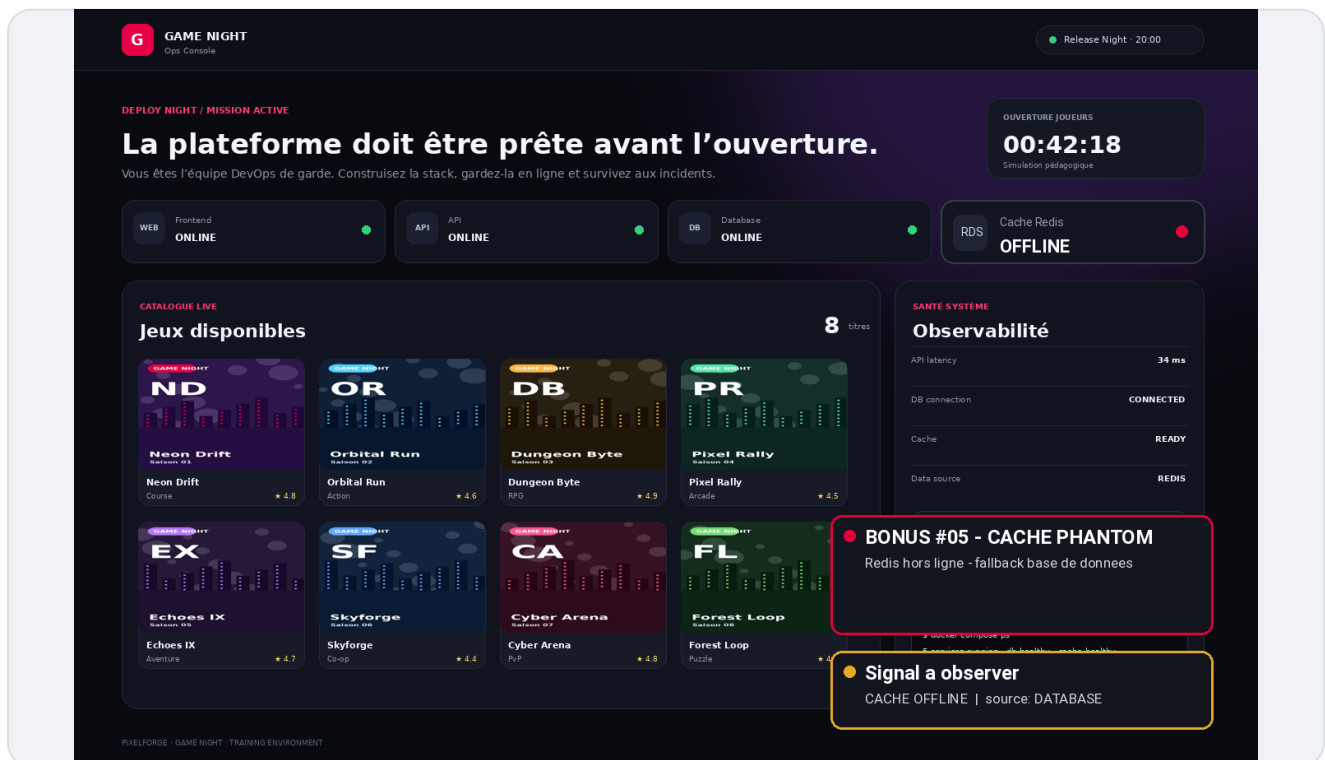
DISCUSSION

Une dégradation sans panne totale doit-elle compter dans votre Change Failure Rate ? Il n'y a pas une réponse universelle : parlez de SLO et d'impact réel.

BONUS 05 • INDICES

Cache Phantom - indices

La bonne équipe ne surveille pas uniquement les erreurs 500. Elle sait reconnaître un système qui fonctionne en mode dégradé.



Visuel d'aide : interface complète conservée, signal cache mis en évidence.

Indice 1

Regardez `/api/status` et la propriété `source` de `/api/games`.

Indice 2

L'API possède un fallback DB : l'absence de panne ne prouve pas que Redis va bien.

Indice 3

Dans Compose, le DNS interne s'appuie sur quel nom ?

BONUS 06 • OPTIONNEL

Access Denied - la config n'est plus acceptée

Dernier bonus : le réseau est bon, la base est prête, mais l'API ne peut toujours pas s'authentifier.

```
./incident 6  
  
# Lisez le message PostgreSQL exact.
```

IMPACT

L'API quitte au démarrage. PostgreSQL est joignable, mais refuse la connexion avec la configuration présentée par le client.

- distinguer erreur réseau, erreur de readiness et erreur d'authentification ;
- comparer la configuration utilisée par l'API avec celle attendue par la DB ;
- corriger la configuration sans hardcoder de secret dans le code ;
- valider avec `./checks/check-incident-06-bonus.sh`.

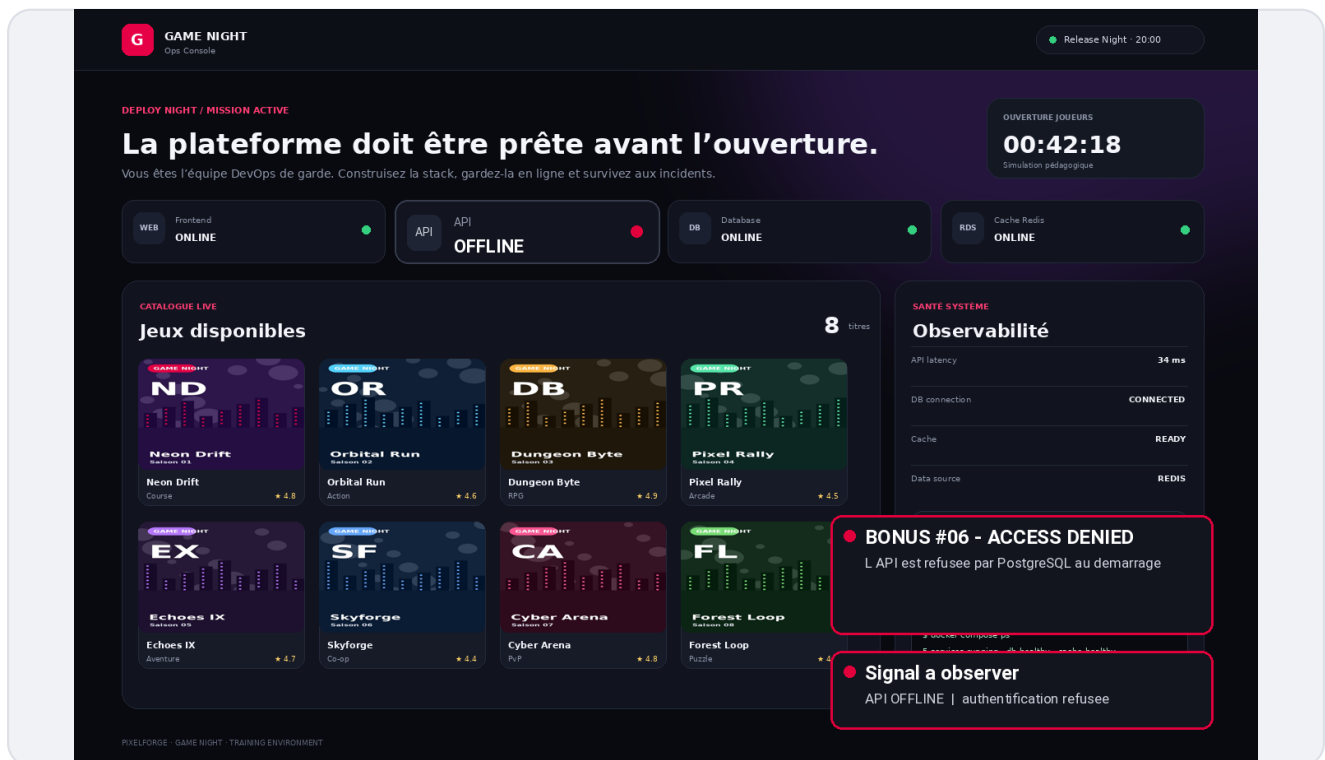
SECURITE

Dans ce TP, les valeurs sont de démonstration. En production, la correction n'est jamais "mettre le vrai mot de passe dans Git".

BONUS 06 • INDICES

Access Denied - indices

Le message d'erreur doit vous permettre de classer la panne avant même de chercher le fichier à modifier.



Visuel d'aide : API hors ligne, base présente, signal d'authentification mis en évidence.

Indice 1

Cherchez "authentication failed" ou équivalent dans les logs API.

Indice 2

La DB et l'API ne lisent pas forcément la même variable de configuration.

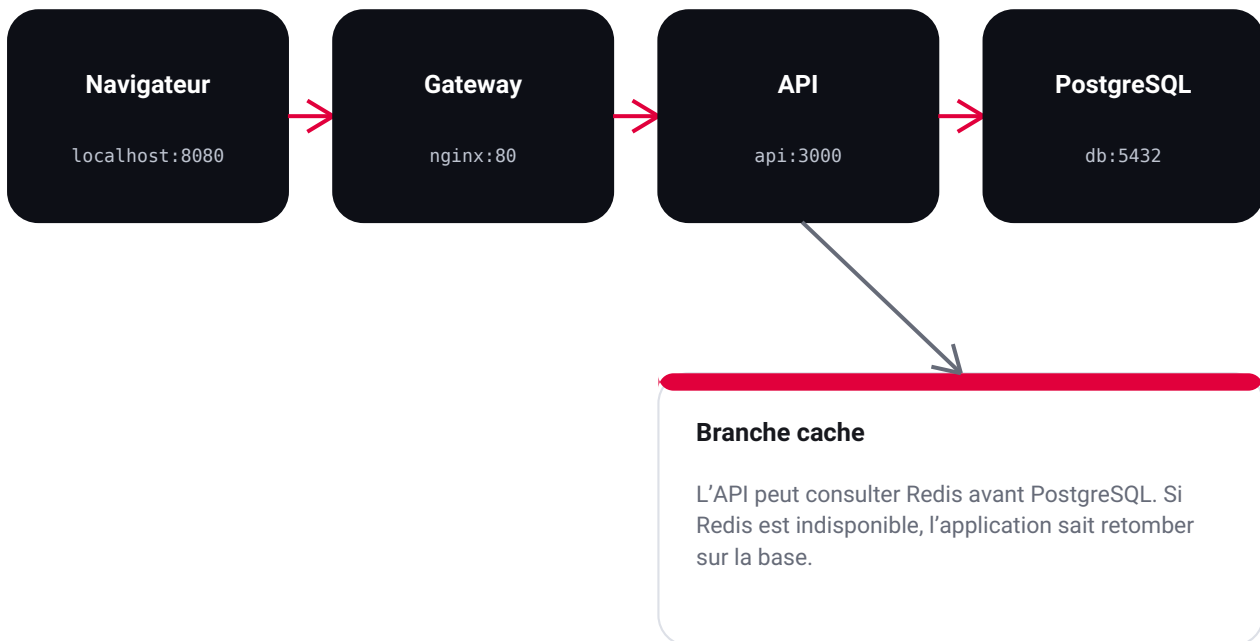
Indice 3

Corrigez la configuration du client qui se connecte - pas le code serveur PostgreSQL.

AVANT DE TERMINER

Dessiner le système dans votre tête

Sans regarder le schéma du début, refaites le trajet d'une requête et expliquez où se trouvent les frontières entre machine hôte et réseau Compose.



Chaque membre doit pouvoir répondre sans lire le support

- Quel port le navigateur utilise-t-il ? Quel port le gateway utilise-t-il vers l'API ?
- Pourquoi localhost change de sens selon l'endroit où la commande s'exécute ?
- Quelle donnée survit à un redémarrage grâce au volume ?
- Que change un healthcheck par rapport à un container simplement "running" ?
- Pourquoi le gateway peut être up alors que la route /api est cassée ?

VALIDATION FINALE

Checklist de fin de garde

Ne cochez pas “ça devrait marcher”. Vérifiez réellement.

- | | |
|--|--|
| ☐ Les Dockerfiles API et frontend utilisent les FROM imposés et construisent sans erreur. | ☐ Le frontend est servi par Nginx et l'API écoute sur son port attendu. |
| ☐ docker compose up -d --build lance la stack. | ☐ Le gateway répond sur le port hôte attendu. |
| ☐ L'API joint PostgreSQL. | ☐ Redis est healthy et l'API peut utiliser le cache. |
| ☐ PostgreSQL possède maintenant un healthcheck fiable. | ☐ L'API attend la DB healthy après l'incident 02. |
| ☐ Les incidents 01, 02 et 03 ont tous un check vert. | ☐ Les 3 Incident Commanders ont tourné. |
| ☐ ops/DORA.md contient les 4 métriques + throughput/stability. | ☐ ops/POSTMORTEM.md est factuel et blameless. |
| ☐ Votre historique Git contient des commits courts et compréhensibles. | ☐ Aucun .env ou état d'incident sensible n'est commité. |

BONUS

Les incidents 04, 05 et 06 sont vraiment facultatifs. Une équipe qui termine proprement le parcours obligatoire a réussi le PLD.

DEBRIEF

15 questions à pouvoir expliquer

Pas de récitation mot à mot. Chaque membre doit être capable de répondre avec un exemple pris dans Game Night.

- 1. Image et container : quelle différence ?
- 2. A quoi servent FROM, WORKDIR, COPY, RUN et CMD ?
- 3. EXPOSE publie-t-il un port sur votre machine ?
- 4. Que signifie HOST:CONTAINER dans ports ?
- 5. Pourquoi Compose est utile ici ?
- 6. build et image : quelle différence dans compose.yaml ?
- 7. Pourquoi api peut joindre db avec un nom de service ?
- 8. Pourquoi localhost a causé Ghost Host ?
- 9. "Started" et "ready" : quelle différence ?
- 10. A quoi sert un healthcheck ?
- 11. Que garantit condition: service_healthy ?
- 12. Pourquoi Bad Gateway peut arriver avec des containers up ?
- 13. Quelles métriques DORA mesurent throughput et stabilité ?
- 14. Pourquoi un post-mortem blameless est utile ?
- 15. Quelle action préventive choisiriez-vous en priorité après ce PLD ?

FIN DE GARDE

20h00 - Game Night est live

Vous n'avez pas seulement lancé des containers. Vous avez construit une stack, observé ses dépendances et rétabli un service sous contrainte.

20:00

Les portes sont ouvertes.

La release est en ligne. La prochaine équipe peut comprendre ce que vous avez laissé.

Vous savez construire

Images Docker reproductibles.

Vous savez diagnostiquer

Logs, états, hypothèses, tests.

Vous savez apprendre

DORA + post-mortem
blameless.

GOOD SHIFT

Si vous avez terminé le parcours obligatoire, gardez le repo propre, poussez vos commits et laissez votre post-mortem lisible. Les bonus sont un plus, pas une condition de réussite.